

Hybrid Intelligent Network Intrusion Detection System

Project Requirements, Architecture, ML Methodology & Evaluation Plan

Proposed title: A Hybrid Machine Learning and Deep Learning Framework for Known and Unknown Network Attack Detection with Explainable AI

Academic stage: 7th Semester Major Project

Primary domain: Artificial Intelligence / Machine Learning / Deep Learning / Cybersecurity

Project objective: Develop and evaluate a network intrusion detection system that combines supervised learning for known attack classification with unsupervised anomaly detection for suspicious or held-out attack behavior, followed by explainable AI and a web dashboard in later phases.

1. Project Overview

The proposed system is a hybrid Network Intrusion Detection System (NIDS). It analyzes network-flow data and determines whether traffic is normal, a known attack, or suspicious/anomalous. The project is designed in stages so that a working ML prototype can be demonstrated first, followed by the more advanced hybrid architecture.

Core idea

Supervised models such as XGBoost and TabTransformer learn labeled attack categories. An Autoencoder learns the structure of normal traffic and produces an anomaly score. A hybrid decision engine combines these signals. SHAP is then used to explain supervised predictions.

Important scope note: an anomaly should not automatically be called a zero-day attack. The project will describe the result as suspicious, anomalous, or an unseen/held-out attack unless additional evidence establishes a specific attack type.

2. Objectives

- Collect and preprocess a recognized network intrusion dataset.
- Build a supervised ML baseline for normal-versus-attack and/or multiclass attack classification.
- Compare Random Forest and XGBoost as initial ML models.
- Implement TabTransformer as the proposed attention-based tabular deep-learning model in a later phase.
- Train an Autoencoder on benign/normal traffic for anomaly detection.
- Design a held-out attack experiment to investigate detection of attack categories excluded from supervised training.
- Combine supervised prediction and anomaly score through a hybrid decision engine.
- Use SHAP to provide interpretable feature-level explanations for model predictions.
- Develop a FastAPI backend and React dashboard after the ML core is validated.
- Evaluate models using precision, recall, F1-score, confusion matrix, false-positive rate and other appropriate metrics.

3. Project Scope

Phase 1 — Evaluation-ready ML prototype: Dataset → preprocessing → Random Forest → XGBoost → metrics → confusion matrix → sample predictions.

Phase 2 — Advanced AI: TabTransformer → Autoencoder → held-out attack experiment → hybrid decision engine.

Phase 3 — Explainability: SHAP-based feature contribution and anomaly analysis.

Phase 4 — Application: FastAPI → PostgreSQL → React dashboard → Docker/cloud deployment.

4. Functional Requirements

FR1 — Dataset Management

- Import network-flow datasets such as CIC-IDS2017 and, if time permits, UNSW-NB15.
- Read labeled network-flow features and attack labels.
- Support combining multiple CSV files where required.

FR2 — Data Preprocessing

- Handle missing, infinite and invalid values.
- Remove duplicates where appropriate.
- Strip/standardize column names.

- Encode categorical values and scale numerical features when required by the model.
- Split data into training, validation and test sets without leaking information from the test set.
- Save preprocessing artifacts so inference uses the same transformations as training.

FR3 — Known Attack Classification

- Train Random Forest as an initial baseline.
- Train XGBoost as a stronger ML baseline.
- Train TabTransformer as the proposed attention-based tabular model in the advanced phase.
- Support binary classification and/or multiclass classification depending on the selected experiment.

FR4 — Anomaly Detection

- Train an Autoencoder primarily on normal/benign traffic.
- Calculate reconstruction error for test samples.
- Determine an anomaly threshold using training/validation data rather than an arbitrary value.
- Flag traffic above the threshold as anomalous/suspicious.

FR5 — Hybrid Decision Engine

- Combine supervised model probabilities/predictions with Autoencoder anomaly scores.
- Produce final categories such as Normal, Known Attack, or Suspicious/Anomalous.
- Record confidence/probability and anomaly score where applicable.

FR6 — Explainable AI

- Generate SHAP explanations for supervised model predictions where supported.
- Display important features contributing to an individual prediction.
- Provide global feature-importance visualizations.

FR7 — Dashboard

- Display traffic totals, normal traffic, known attacks and anomalies.
- Show attack distribution and recent alerts.
- Allow a user to inspect a prediction and its explanation.
- Support CSV upload and batch prediction in the first application version.

5. Proposed System Architecture

Overall pipeline

Network Traffic / Dataset → Data Collection → Preprocessing → Feature Engineering → XGBoost / TabTransformer + Autoencoder → Hybrid Decision Engine → Final Classification → SHAP Explanation → FastAPI → React Dashboard

Detection architecture

Network Flow → Preprocessing → Feature Vector → [TabTransformer / XGBoost] AND [Autoencoder] → Supervised Prediction + Anomaly Score → Hybrid Decision Engine → Normal / Known Attack / Suspicious-Unseen → SHAP → Alert

6. Process Flowchart

Training flow

CIC-IDS2017 / UNSW-NB15 → Cleaning → Feature Engineering → Train/Validation/Test Split → XGBoost Training + TabTransformer Training → Model Evaluation → Autoencoder Training on Normal Data → Anomaly Threshold Selection → Hybrid Decision Engine → Final Evaluation

Detection flow

New Network Flow → Preprocessing → Feature Vector → Supervised Model + Autoencoder → Attack Probability + Anomaly Score → Hybrid Decision → Final Classification → Explainability → Alert / Report

7. Dataset Requirements

Primary candidate: CIC-IDS2017. It is a recognized intrusion-detection dataset containing labeled network traffic and flow features suitable for supervised ML experiments.

Alternative/secondary dataset: UNSW-NB15, if time and compute resources permit.

Recommended initial task: start with a manageable subset/file to establish the complete pipeline, then scale to combined data.

Held-out attack experiment: select one attack category and exclude it from supervised training. Evaluate whether the anomaly detector can flag that held-out category as suspicious.

8. ML/DL Model Requirements

Model	Role	Phase
Random Forest	Initial ML baseline	Phase 1
XGBoost	Strong supervised ML baseline	Phase 1
TabTransformer	Attention-based tabular classification	Phase 2
Autoencoder	Unsupervised anomaly detection	Phase 2
SHAP	Explainability, not a classifier	Phase 3

9. Evaluation Requirements

The evaluation must use actual experimental results. Do not insert assumed or fabricated accuracy values.

Metric	Use
Accuracy	Overall correctness; should not be the only metric for imbalanced security data.
Precision	How many predicted attacks are actually attacks; relevant to false alarms.
Recall	How many actual attacks are detected; important for security.
F1-score	Balances precision and recall.
Confusion Matrix	Shows class-wise errors and misclassification patterns.
False Positive Rate	Measures normal traffic incorrectly flagged as malicious.
False Negative Rate	Measures attacks missed by the detector.
ROC-AUC	Optional/appropriate for supported binary or multiclass evaluation.
Inference Time	Useful for assessing practical detection performance.

10. Experimental Plan

1. Train and evaluate Random Forest as the first baseline.

2. Train and evaluate XGBoost using the same evaluation protocol.
3. Compare precision, recall, F1-score and confusion matrices.
4. Implement TabTransformer after the classical ML baseline is stable.
5. Train the Autoencoder on normal traffic and determine an anomaly threshold from validation data.
6. Create a held-out attack experiment to test unseen/held-out attack detection.
7. Combine supervised predictions and anomaly scores in the hybrid decision engine.
8. Add SHAP explanations.
9. Integrate the final model with FastAPI and the React dashboard.

11. Immediate Requirement for Next-Week Faculty Evaluation

The immediate target is **not** the complete production system. The target is a working ML prototype demonstrating that ML methods have been applied to the cybersecurity problem.

- Dataset loaded and inspected.
- Data cleaning and preprocessing demonstrated.
- Random Forest trained and evaluated.
- XGBoost trained and evaluated.
- Precision, recall, F1-score and accuracy reported from actual test results.
- Confusion matrix displayed.
- Class/attack distribution visualized.
- A few unseen test samples passed through the trained model and predictions displayed.
- Short comparison between Random Forest and XGBoost.
- Explain that TabTransformer + Autoencoder + hybrid decision engine are the next phase of the proposed system.

Suggested explanation to faculty

“In the first phase, we implemented supervised machine learning for network intrusion detection using Random Forest and XGBoost. We are evaluating the models using precision, recall, F1-score and confusion matrices. The proposed next phase adds TabTransformer for attention-based classification and an Autoencoder for anomaly detection of held-out or suspicious attack behavior, followed by hybrid decision fusion and Explainable AI.”

12. Software Requirements

- Python 3.x
- Jupyter Notebook, Google Colab, or VS Code
- Pandas, NumPy
- Scikit-learn
- XGBoost
- PyTorch or TensorFlow for TabTransformer/Autoencoder
- SHAP
- Matplotlib / Seaborn for visualizations
- FastAPI + Uvicorn for backend
- React + Tailwind CSS for frontend

- PostgreSQL for persistent storage
- Git/GitHub for version control
- Docker for packaging/deployment

13. Hardware Requirements

- Minimum: modern Intel i5 / Ryzen 5 class CPU, 8 GB RAM and sufficient SSD storage.
- Recommended: 16 GB RAM and SSD.
- NVIDIA GPU with CUDA is useful for deep-learning training but is not mandatory for the initial Random Forest/XGBoost prototype.

14. Non-Functional Requirements

- Reliability: invalid input should be handled without crashing the application.
- Performance: individual prediction requests should return within a reasonable response time.
- Maintainability: separate preprocessing, training, inference, API and UI modules.
- Reproducibility: record dataset version, preprocessing configuration, model parameters and evaluation results.
- Scalability: allow additional datasets and models to be added.
- Security: validate uploaded files and avoid executing uploaded content.
- Usability: dashboard results should be understandable to a security analyst/student evaluator.

15. Major Project Modules

Module	Description
1. Dataset & Preprocessing	Load, clean, encode, scale and split network-flow data.
2. Feature Engineering	Select/transform relevant network-flow features.
3. ML Classification	Random Forest and XGBoost baseline experiments.
4. TabTransformer	Attention-based tabular attack classification.
5. Autoencoder	Normal-traffic reconstruction and anomaly detection.
6. Hybrid Decision Engine	Combine supervised prediction and anomaly score.
7. Explainable AI	SHAP feature-level explanations.
8. Web Application	FastAPI backend, React dashboard and optional PostgreSQL.

16. Recommended Development Roadmap

Week/Evaluation 1: Dataset + preprocessing + Random Forest + XGBoost + metrics + confusion matrix.

Phase 2: TabTransformer + model comparison.

Phase 3: Autoencoder + anomaly threshold + held-out attack experiment.

Phase 4: Hybrid decision engine + SHAP.

Phase 5: FastAPI + React + PostgreSQL dashboard.

Phase 6: Docker/cloud deployment, final experiments, report, paper and viva preparation.

17. Final Proposed System

The final system will combine **XGBoost/TabTransformer** for supervised known-attack classification with an **Autoencoder** for anomaly detection. A **hybrid decision engine** will combine their outputs, while **SHAP** will provide interpretability. The application layer will expose the model through **FastAPI** and visualize alerts through a **React dashboard**.

Central research question: Can combining supervised attention-based classification with unsupervised anomaly detection improve the detection of known and held-out attack categories while maintaining an acceptable false-positive rate?

Prepared as a project planning and evaluation document. Actual model performance must be established experimentally on the selected dataset.